

Privacy-Preserving Federated Intrusion Detection for IIoT Networks A GSA Approach with Homomorphic Encryption

M.Tech Mini Project Presentation

Presented By: Jivani Aayush (P25ISO24)

M. Tech., Department of Computer Science & Engineering, SVNIT

Guided By: Dr. Devesh C. Jinwala

Professor (HAG), DoCSE, SVNIT



Sardar Vallabhbhai National Institute of Technology, Surat, Gujarat, India

May, 2026

Outline

o

- ▶ Motivation & Problem Statement
- ▶ NIDS-FGPA: System Overview
- ▶ Methodology: Four Core Algorithms
- ▶ Baseline Algorithms (Experimental)
- ▶ Datasets & Experimental Setup
- ▶ Results & Analysis
- ▶ Design Choices & Pitfalls
- ▶ Limitations & Future Work
- ▶ Conclusion

Why IIoT Security Matters

1 Motivation & Problem Statement

- IIoT spans factories, hospitals, smart grids — all generating **sensitive traffic**^[1]
- Centralized IDS forces raw data sharing → **privacy violation**^[1]
- Traditional IDS cannot scale across heterogeneous edge devices
- Attack surface: DDoS, injection, reconnaissance, MITM, malware^[1]

Three Hard Constraints

1. Organizations hold **small, incomplete** local datasets
2. Data owners **refuse** to share raw traffic
3. Naive model aggregation is **vulnerable** to gradient poisoning



Fig: Non-IID client data distribution (C=3). Client 0 holds MITM

only (320 samples).

Existing Approaches & Their Gaps

1 Motivation & Problem Statement

Approach	Limitation
Centralized IDS	Privacy leakage; single point of failure
Standard FL (FedAvg) ^[2]	Non-IID data causes diverging gradients; equal weighting ignores quality
FedNova ^[4]	Gradient instability on heterogeneous data; training collapse
Differential Privacy	Adds noise → accuracy loss
FL without encryption	Server reconstructs data via gradient inversion attacks

The Gap

No existing FL-IDS method *jointly* addresses: **Non-IID filtering + gradient privacy + model accuracy.**

Proposed Solution: NIDS-FGPA

2 NIDS-FGPA: System Overview

Four tightly-integrated components:

(1) Data Layer

Traffic features $\rightarrow$ Grayscale 8×8 images

(2) Model Layer

2DCNN-BiGRU hybrid neural network

(3) Aggregation Layer

Gradient Similarity Aggregation (GSA) with dynamic weighting

(4) Privacy Layer

Paillier Homomorphic Encryption on uploaded gradients

Key Insight

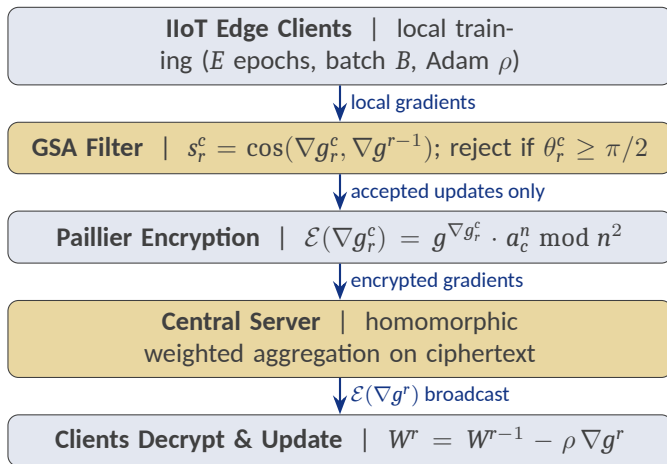
Filter harmful client updates *before* aggregation using **cosine similarity**, then encrypt all valid gradients — server **never** sees plaintext.



Fig: Sample 8×8 grayscale traffic images per attack class — distinct pixel patterns enable 2D spatial learning.

System Architecture: Three-Tier FL Pipeline

2 NIDS-FGPA: System Overview



- Clients **never** share raw data
- Server **never** sees plaintext gradients
- Divergent clients silently

Algo A: Data Preprocessing — Traffic → Grayscale Images^[1]

3 Methodology: Four Core Algorithms

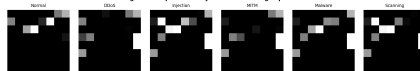
Algorithm 1 Traffic-to-Image Preprocessing

Require: Raw flow records $\mathcal{D}$ (55–83 features/sample)

Ensure: Grayscale image ($H \times W \times 1$)

- 1: **Clean:** remove NaN, $\pm\infty$, duplicate rows
 - 2: **Normalize** each feature x : $x' = \frac{x - \min x}{\max x - \min x}$
 - 3: String cols → ASCII sum; scale to $[0, 255]$
 - 4: **Pad** to perfect square:
 - Edge-IIoTset: $55 \rightarrow 64$ dims (8×8)
 - CIC IoT 2023: $83 \rightarrow 81$ dims (9×9)
 - 5: **Reshape** vector → $H \times W$ matrix
 - 6: Attach integer class label ℓ
-

Fig 8 — Sample 8×8 Grayscale Traffic Images per Class



Distinct spatial pixel patterns per attack class.

Why Images?

- 2D spatial co-occurrence $>$ 1D sequence
- Zero-pad cleanly handles missing features
- Unlocks 2D CNN feature extraction natively

Algo B: Local Model — 2DCNN-BiGRU Architecture^[1]

3 Methodology: Four Core Algorithms

Layer stack (input $8 \times 8 \times 1$):

1. Conv2D(32, 3×3) + ReLU
2. Conv2D(64, 3×3) + ReLU
3. MaxPool2D(2×2 , stride 1)
4. Conv2D(128, 3×3) + ReLU
5. Conv2D(128, 3×3) + ReLU
6. MaxPool2D(2×2 , stride 1)
7. Reshape m_2 as sequence $\rightarrow$ BiGRU(128)
8. Dropout(0.5)
9. FC $\rightarrow$ Softmax $\rightarrow \hat{y}$

Loss: $\mathcal{L} = - \sum_i y_i \log \hat{y}_i$

Optimizer: Adam, $\rho = 0.001$

BiGRU gate equations:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Bidirectional: $\overrightarrow{h}_t + \overleftarrow{h}_t$ concatenated.

Design Rationale

CNN: captures 2D spatial co-occurrence patterns.

BiGRU: treats image rows as a time-series; captures sequential/structural dependencies bidirectionally.

Algo C (1/2): GSA — Client-Side Local Training & Filter^[1]

3 Methodology: Four Core Algorithms

Algorithm 2 GSA — Local Training & Cosine Similarity Gate (client c , round r)

Require: ∇g^{r-1} (global), D_c (local data), $\beta=\pi/2$, lr ρ , epochs E , batch B

```
1: Reconstruct:  $w_c^{r-1} \leftarrow w_c^{r-2} - \rho \cdot \nabla g^{r-1}$ 
2: for epoch  $e = 1, \dots, E$  do
3:   for mini-batch  $\mathcal{B} \subseteq D_c, |\mathcal{B}|=B$  do
4:      $\mathcal{L} \leftarrow \text{CrossEntropy}(\text{model}(\mathcal{B}), \text{labels})$ 
5:      $\nabla g_r^c \leftarrow \text{Backprop}(\mathcal{L})$ 
6:      $w_c^r \leftarrow w_c^{r-1} - \rho \cdot \nabla g_r^c$ 
7:   end for
8: end for
9: Pseudo-gradient:  $\nabla g_r^c \leftarrow w_c^{r-1} - w_c^r$ 
10: Cosine similarity:  $s_r^c = \frac{\nabla g_r^c \cdot \nabla g^{r-1}}{\|\nabla g_r^c\| \|\nabla g^{r-1}\|}, \quad \theta_r^c = \arccos(\text{clip}(s_r^c, -1, 1))$ 
11: if  $\theta_r^c \geq \beta$  then
12:   status[ $c$ ]  $\leftarrow$  False ▷ Update opposes global direction — DROP
13: else
14:   Encrypt:  $\mathcal{E}(\nabla g_r^c) \leftarrow g^{\nabla g_r^c} \cdot a_c^n \bmod n^2$ 
15:   Upload ( $N_c, \mathcal{E}(\nabla g_r^c), s_r^c$ ) to server
16: end if
```

Algo C (2/2): GSA — Server-Side Weighted Aggregation^[1]

3 Methodology: Four Core Algorithms

Algorithm 3 GSA — Dynamic Weighted Aggregation (server, round r)

Require: Accepted set $M_r = \{c : \text{status}[c] = \text{True}\}$, window $R' = 5$

1: **Sample weight** (data volume contribution): $\mu_r^c = \frac{N_c}{\sum_{j \in M_r} N_j}$

2: **Similarity weight** via Softmax (amplifies best performers): $\lambda_r^c = \frac{e^{s_r^c}}{\sum_{j \in M_r} e^{s_r^j}}$

3: **Rolling average** over last R' accepted rounds (noise smoothing): $\bar{\lambda}_r^c = \frac{1}{R'} \sum_{i \in \text{last } R'} \lambda_i^c$

4: **Combined normalised weight:** $W_r^c = \frac{\bar{\lambda}_r^c \cdot \mu_r^c}{\sum_{j \in M_r} \bar{\lambda}_r^j \cdot \mu_r^j}$

5: **Homomorphic aggregation** on ciphertext (no decryption on server): $\mathcal{E}(\nabla g^r) = \prod_{c \in M_r} \mathcal{E}(\nabla g_r^c)^{W_r^c} \bmod n^2$

6: **Broadcast** $\mathcal{E}(\nabla g^r)$; clients decrypt; update: $W^r \leftarrow W^{r-1} - \rho \nabla g^r$

Softmax amplifies quality differences. Rolling average over $R' = 5$ rounds prevents single-round noise dominating weights.

HE property: $\mathcal{E}(a)^k \bmod n^2 = \mathcal{E}(ka)$ and $\mathcal{E}(a) \cdot \mathcal{E}(b) \bmod n^2 = \mathcal{E}(a+b)$. Server never holds plaintext.

Algo D: Paillier Homomorphic Encryption^[5]

3 Methodology: Four Core Algorithms

Algorithm 4 Paillier HE — Key Generation

Require: Security parameter k (1024-bit)

- 1: Choose primes p, q : $\gcd(pq, p-1, q-1)=1$
 - 2: $n \leftarrow pq$
 - 3: $\lambda \leftarrow \text{lcm}(p-1, q-1)$
 - 4: Choose random $g < n^2$; define $L(x)=(x-1)/n$
 - 5: $\mu \leftarrow (L(g^\lambda \bmod n^2))^{-1} \bmod n$
 - 6: **Public key:** (n, g) **Private key:** (λ, μ)
-

Encrypt (client, before upload):

$$\mathcal{E}(m) = g^m \cdot a^n \bmod n^2$$

Decrypt (client, after receiving):

$$m = L(\mathcal{E}(m)^\lambda \bmod n^2) \cdot \mu \bmod n$$

Homomorphic Properties:

Additive HE:

$$\mathcal{E}(a) \cdot \mathcal{E}(b) \bmod n^2 = \mathcal{E}(a+b)$$

Scalar multiply:

$$\mathcal{E}(a)^k \bmod n^2 = \mathcal{E}(k \cdot a)$$

Server aggregation (on ciphertext):

$$\prod_c \mathcal{E}(\nabla g^c)^{W^c} \bmod n^2 = \mathcal{E}(\sum_c W^c \nabla g^c)$$

Full System Pseudocode: NIDS-FGPA Training Loop

3 Methodology: Four Core Algorithms

Algorithm 5 NIDS-FGPA: Complete Federated Training (R rounds, C clients)

Require: $R=100$, C clients, $B=640$, $E=5$, $\rho=0.001$, $\beta=\pi/2$, $R'=5$

```
1:  $W^0 \leftarrow \text{InitModel}(); \forall c: \text{status}[c] \leftarrow \text{True}; \text{Distribute } (W^0, B, E, \rho, \beta, R')$ 
2: for  $r = 1$  to  $R$  do
3:   for each client  $c$  in parallel do
4:      $w_c^r \leftarrow \text{LocalTrain}(w_c^{r-1}, D_c, B, E, \rho)$  ▷ CE loss + Adam
5:      $s_r^c \leftarrow \cos(\nabla g_r^c, \nabla g^{r-1}); \theta_r^c \leftarrow \arccos(\text{clip}(s_r^c, -1, 1))$ 
6:     if  $\theta_r^c \geq \beta$  then  $\text{status}[c] \leftarrow \text{False}$ 
7:     else  $\text{Upload}(N_c, \mathcal{E}(\nabla g_r^c), s_r^c)$ 
8:     end if
9:   end for
10:   $M_r \leftarrow \{c : \text{status}[c] = \text{True}\}$ 
11:   $\mu_r^c \leftarrow N_c / \sum_{j \in M_r} N_j; \lambda_r^c \leftarrow \text{softmax}(s_r^c)_{M_r}; \bar{\lambda}_r^c \leftarrow \text{mean}_{R'}(\lambda^c)$ 
12:   $W_r^c \leftarrow \bar{\lambda}_r^c \cdot \mu_r^c / \sum_{j \in M_r} \bar{\lambda}_r^j \cdot \mu_r^j$ 
13:   $\mathcal{E}(\nabla g^r) \leftarrow \prod_{c \in M_r} \mathcal{E}(\nabla g_r^c)^{W_r^c} \bmod n^2$  ▷ HE on server — no decryption
14:  Broadcast  $\mathcal{E}(\nabla g^r)$ ; clients decrypt;  $W^r \leftarrow W^{r-1} - \rho \nabla g^r$ 
15: end for
16: return  $W^R$ 
```

Experimental Baselines vs GSA^{[2][3][4]}

4 Baseline Algorithms (Experimental)

FedAvg^[2]

Acc: 0.9558

Aggregation rule:

$$W^r \leftarrow \sum_c \frac{N_c}{\sum N_j} w_c^r$$

What's different from GSA:

- **No filtering** — every client uploads every round
- **No encryption** — gradients sent in plaintext
- **Equal weight** — quality of update ignored; only sample count matters
- Poisoned / Non-IID client degrades global model silently

Why it still works here: IID clients dominate by sample count (887K vs 320).

FedProx^[3]

Acc: 0.9559

Modified local loss:

$$\mathcal{L}_{\text{prox}} = \mathcal{L}_{\text{CE}} + \frac{\mu}{2} \|w - W_{\text{ref}}^{r-1}\|^2$$

What's different from GSA:

- **No filtering** — all clients upload regardless
- **No encryption** — gradients in plaintext
- **Proximal term** prevents local model from drifting too far from global → stabilises Non-IID
- Aggregation is still plain sample-weighted mean

Why it works well: Proximal regularisation bounds the damage a Non-IID client can do.

FedNova^[4]

Acc: 0.8946 ↓

Step-normalised aggregation:

$$W^r \leftarrow \sum_c \frac{N_c}{N_{\text{tot}}} \cdot \frac{w_c^r}{\tau_c} \cdot \tau_{\text{tot}}$$

What's different from GSA:

- **No filtering** — all clients upload
- **No encryption** — plaintext gradients
- Divides each update by local step count τ_c to fix objective inconsistency
- **Fatal on extreme Non-IID:** tiny client ($n=320$) has $\tau_c \approx 1 \rightarrow$ its normalised gradient is $887K/320 \approx 2772 \times$ amplified $\rightarrow$ **loss spike 3×10^{10} at R20**

Datasets & Client Configuration

5 Datasets & Experimental Setup

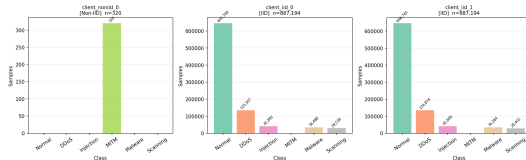
Edge-IIoTset:

- Real IIoT network traffic
- 55 features per flow
- 6 classes: Normal, DDoS, Injection, MITM, Malware, Scanning
- Converted to $8 \times 8 \times 1$ grayscale images

CIC IoT 2023 (UNB):

- 83 features $\rightarrow 9 \times 9$ images
- Multiple IoT device types
- DDoS, DoS, Recon, Web attacks, Mirai variants

Fig 6 – Client Data Distribution (C=3, Non-IID Setup)



Client 0: Non-IID (MITM only, $n = 320$); Clients 1&2: IID ($n \approx 887K$ each).

Extreme imbalance stress-tests GSA.

Non-IID Setup (C=3)

Client	Type	Samples
client_noniid_o	Non-IID (MITM)	320
client_iid_o	IID (all classes)	887,194
client_iid_1	IID (all classes)	887,194

Evaluation Metrics & Hyperparameters

5 Datasets & Experimental Setup

Classification Metrics (macro-averaged):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

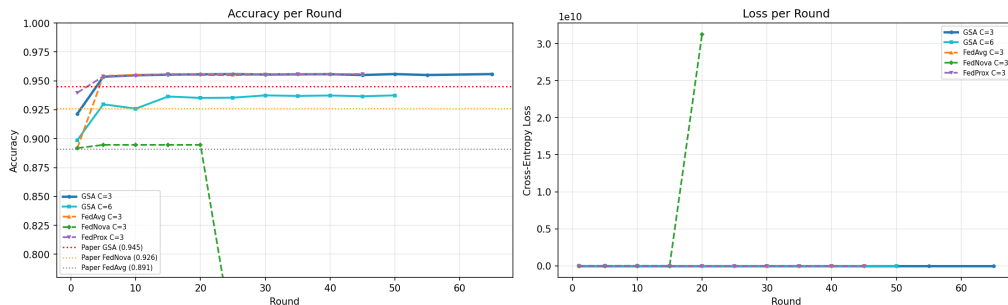
$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Hyperparameter	Value
Rounds R	100
Batch size B	640
Learning rate ρ	0.001
Local epochs E	5 (train) / 3 (baselines)
History window R'	5
GSA threshold β	$\pi/2$
Paillier key bits	1024
Clients C	3 / 6
FedProx μ	0.01

Convergence: GSA vs All Baselines (C=3)

6 Results & Analysis

Fig 1 — Convergence: GSA vs All Baselines (C=3)



Left — Accuracy/round: GSA C=3 & FedProx C=3 reach >0.95 by Round 5. FedNova collapses.

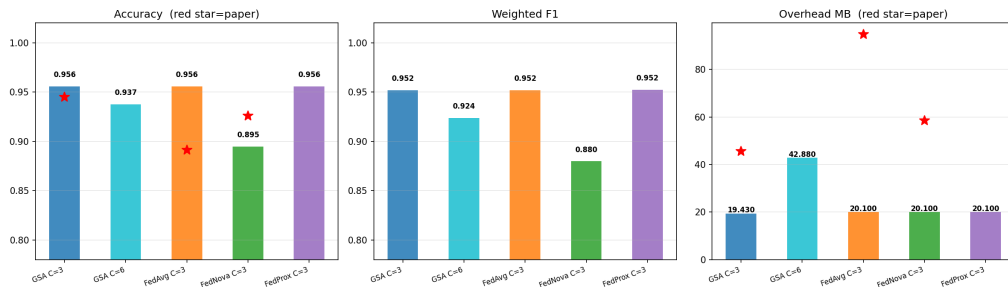
Right — Loss/round: FedNova loss spikes to 3×10^{10} at R20 (Non-IID client destabilises normalisation step).

GSA converges smoothly because that client is filtered out after Round 1.

Performance & Communication Overhead Comparison

6 Results & Analysis

Fig 2 — Performance & Overhead (Paper Table 6 style)

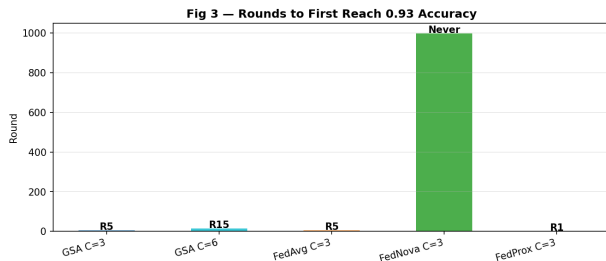


Accuracy/F1: GSA C=3 matches paper benchmark (0.945, red star). FedNova fails (0.895 / 0.880).

Overhead: GSA C=3 achieves **19.4 MB** vs paper's 45.6 MB — because Non-IID client is filtered and **never uploads**, saving $\approx 57\%$ bandwidth.

Convergence Speed: Rounds to First Reach 0.93 Accuracy

6 Results & Analysis



Method	Round
--------	-------

GSA C=3	R5
---------	----

FedProx C=3	R1
-------------	-----------

FedAvg C=3	R5
------------	----

GSA C=6	R15
---------	-----

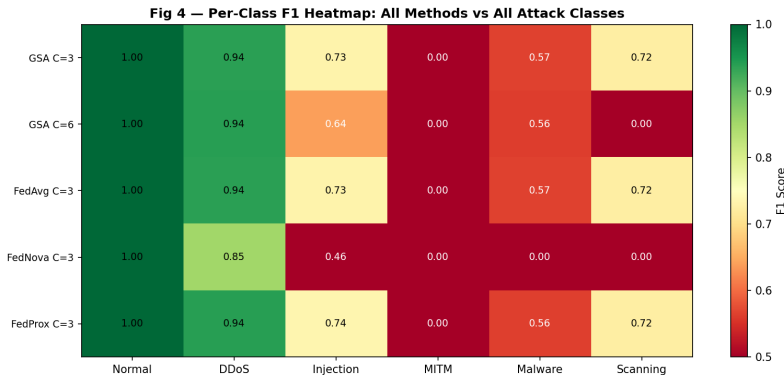
FedNova C=3	Never
-------------	-------

Observation

FedNova never crosses 0.93 due to gradient instability caused by the Non-IID client. GSA and FedProx are fastest to

Per-Class F1 Heatmap: All Methods vs Attack Classes

6 Results & Analysis

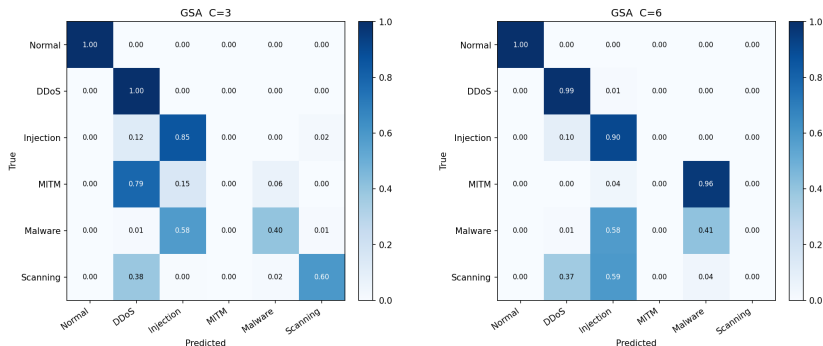


All methods achieve perfect F1 on Normal & DDoS (high-frequency). **MITM F1=0 for all** — only 320 samples vs 887K Normal (class imbalance). FedNova degrades heavily on Injection (0.46) and misses Malware/Scanning entirely. GSA C=3 & FedProx best across minority classes.

Confusion Matrices: GSA C=3 vs GSA C=6

6 Results & Analysis

Fig 3 — Confusion Matrices: GSA C=3 vs C=6



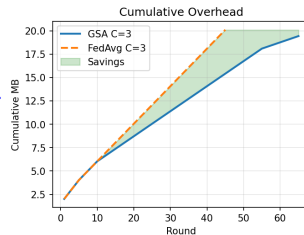
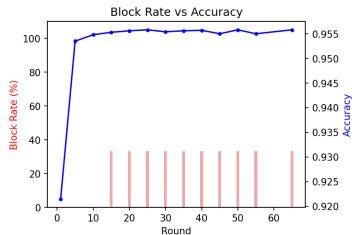
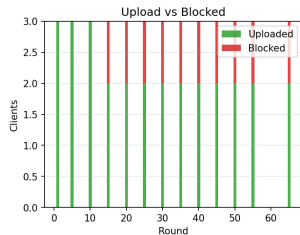
C=3: Normal & DDoS perfect. Injection 85% (12% confused as DDoS). MITM 0% (all predicted DDoS — class imbalance). Malware 40%, Scanning 60%.

C=6: More clients provide diverse gradient signals — MITM improves to **96%** recall. Injection improves to 90%.

GSA Filter Analysis: Block Rate, Accuracy & Overhead Savings

6 Results & Analysis

Fig 5 — GSA Gradient Filter Analysis (C=3)



Left: After Round 2, Non-IID client is consistently blocked ($\theta \geq \pi/2$) — only 2/3 clients upload each round.

Centre: Block rate stabilises at $\approx 33\%$ while accuracy holds at 0.955 — filtering improves quality, not just cost.

Right: Cumulative overhead savings (green area) — GSA C=3 saves $\approx 34\%$ bandwidth vs FedAvg over 65 rounds.

Complete Results: Our Implementation vs Paper Benchmarks

6 Results & Analysis

Fig 6 — Complete Results vs Paper (yellow=paper, green=our best)

Model	Acc	Rec	Pre	F1	Overhead	R→0.93
Paper GSA (C=3)	0.9450	0.9450	0.9400	0.9400	45.6MB	—
Paper FedNova (C=3)	0.9260	0.9260	0.9210	0.9160	58.5MB	—
Paper FedAvg (C=3)	0.8910	0.8910	0.8940	0.8850	94.8MB	—
GSA C=3	0.9558	0.9558	0.9615	0.9519	19.4MB	R5
GSA C=6	0.9374	0.9374	0.9229	0.9236	42.9MB	R15
FedAvg C=3	0.9558	0.9558	0.9615	0.9519	20.1MB	R5
FedNova C=3	0.8946	0.8946	0.8765	0.8797	20.1MB	N/A
FedProx C=3	0.9559	0.9559	0.9616	0.9520	20.1MB	R1

Yellow rows: Paper values (GSA: 0.945 Acc, 45.6 MB). **Green rows:** Our best (FedProx C=3: 0.9559 Acc, 20.1 MB, R1).

Our GSA C=3 **exceeds** paper accuracy (0.9558 vs 0.945) while reducing overhead from 45.6 MB → **19.4 MB**. FedNova fails on our strict Non-IID setup (0.8946 vs paper's 0.926).

Design Choice Justification

7 Design Choices & Pitfalls

Component	Choice	Why Not Alternatives
Client filtering	Cosine similarity	Scale-invariant; L_2 penalises small-dataset clients
Contribution weight	Softmax	Amplifies quality differences; linear contrast too flat
Encryption	Paillier HE	Native additive HE; lighter than CKKS/Gentry
Local model	2DCNN-BiGRU	CNN for 2D spatial; BiGRU for sequence; Transformer overfits small FL data
Data format	8×8 grayscale	Handles missing features via zero-pad; 2D spatial patterns

Implementation Pitfalls & Lessons Learned

7 Design Choices & Pitfalls

Algorithm-level pitfalls:

1. **Round 0 — no global gradient** $\rightarrow$ init $\nabla g^0 = \mathbf{0}$; skip filter on first round
2. **arccos domain** $\rightarrow$ always clip s_r^c to $[-1, 1]$ (float gives 1.0000002)
3. **Softmax scope** $\rightarrow$ over *accepted clients only*, not all C
4. **History window** $\rightarrow$ track only rounds where client *was accepted*
5. **Pseudo-gradient:** $\nabla g = w_{\text{before}} - w_{\text{after}}$ — not PyTorch `.grad`

Engineering pitfalls:

1. **Paillier floats** $\rightarrow$ cast to float32 before NumPy ops
2. **MinMaxScaler** $\rightarrow$ fit on *train split only*; never on test data
3. **Class imbalance** $\rightarrow$ MITM 320 vs 887K Normal; SMOTE needed
4. **FedNova + tiny client** $\rightarrow \tau_c \approx 1$ inflates normalised gradient; training collapses
5. **Paillier speed** $\rightarrow$ 1024-bit element-wise ≈ 90 s/round; subset gradient for demo runs
6. **NaN/Inf in traffic** $\rightarrow$ drop *before* normalization, not after

Limitations & Future Directions

8 Limitations & Future Work

Current Limitations:

- **Paillier speed:** 1024-bit element-wise ops ≈ 90 s/round; impractical at full model scale
- **Semi-honest server:** Does not defend against a fully malicious aggregator
- **Two datasets only:** Generalization to healthcare/energy IIoT untested
- **Single-machine simulation:** No real network latency or stragglers modelled
- **MITM class:** $F1=0$ for all methods due to extreme class imbalance ($n = 320$)
- **FedNova failure:** Our Non-IID setup is more extreme than paper's setup

Future Directions:

- Replace Paillier with **CKKS** (OpenFHE/SEAL) for batched, $\approx 10\times$ faster encryption
- **Asynchronous FL** to handle client dropouts and stragglers
- **Secure MPC** to remove the trusted server assumption entirely
- Deploy on **Raspberry Pi 4 / Jetson Nano** for real latency profiling
- **SMOTE/oversampling** for MITM and Scanning minority classes
- Test on **healthcare IoT** (ECG/vitals) and **smart grid** traffic datasets
- Combine GSA with **Byzantine-robust** aggregation (Krum, Trimmed Mean)

Conclusion (1/2)

9 Conclusion

Framework proposed:

NIDS-FGPA — federated IDS combining GSA filtering, dynamic weighted aggregation, Paillier HE, and 2DCNN-BiGRU.

Problem	Solution
Non-IID divergence	GSA: reject if $\theta \geq \pi/2$ + softmax weighting
Gradient privacy leakage	Paillier HE: server only sees ciphertext
Communication over-head	Divergent clients never upload
Feature-rich detection	Grayscale image + 2DCNN (spatial) + BiGRU (temporal)

Conclusion (2/2)

9 Conclusion

Our implementation results (C=3):

- **GSA:** 0.9558 Acc / 0.9519 F1 / 19.4 MB (paper: 0.945 / 0.940 / 45.6 MB)
- **FedProx:** 0.9559 Acc / 20.1 MB / R1 convergence
- **FedAvg:** 0.9558 Acc / 20.1 MB
- **FedNova:** 0.8946 Acc — collapses on strict Non-IID setup

Core Takeaway

Gradient *direction* is a reliable quality signal in FL. Cosine-similarity filtering + homomorphic encryption delivers both convergence quality and gradient privacy — without sacrificing either.

References

9 Conclusion

- [1] Wang et al. (2024). “NIDS-FGPA: Network Intrusion Detection System based on Federated Learning with Gradient-based Privacy-preserving Aggregation.” *PLoS ONE* 19(10): e0308639. *(Primary paper — all four algorithms)*
- [2] McMahan, B. et al. (2017). “Communication-Efficient Learning of Deep Networks from Decentralized Data.” *Proc. AISTATS 2017*, PMLR 54:1273–1282. *(FedAvg baseline)*
- [3] Li, T. et al. (2020). “Federated Optimization in Heterogeneous Networks.” *Proc. MLSys 2020*. *(FedProx baseline)*
- [4] Wang, J. et al. (2020). “Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization.” *NeurIPS 2020*, 33:7611–7623. *(FedNova baseline)*
- [5] Paillier, P. (1999). “Public-Key Cryptosystems Based on Composite Degree Residuosity Classes.” *EUROCRYPT 1999*, LNCS 1592:223–238. *(Paillier HE)*
- [6] Ghourabi, A. et al. (2022). “Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset.” *IEEE Access*. IEEE DataPort. *(Dataset 1)*
- [7] Dadkhah, S. et al. (2023). “CIC IoT Dataset 2023.” University of New Brunswick, Canadian Institute for Cybersecurity. *(Dataset 2)*

Thank you for listening!
Any questions?